WriteOps

Write. Organize. Build. Publish.

Markdown → pandoc → PDF

Powerful Conversion

FOCUS ON YOUR IDEAS.
WE'LL HANDLE THE REST.

Your words. Your way. Your book.

Your Book Title Here

Author Name

Contents

Preface 4

1 Writing Your Book with Markdown 5
- 1.1 Basic Formatting . . . 5
  - 1.1.1 Code Blocks . . . 5
  - 1.1.2 Mathematical Formulas . . . 5
  - 1.1.3 Diagrams with Mermaid . . . 5
  - 1.1.4 Adding Images . . . 6

2 Automation and the Build Pipeline 8
- 2.1 The Build Script . . . 8
- 2.2 Running the Build . . . 8
  - 2.2.1 1. Local Build . . . 8
  - 2.2.2 2. Docker Build . . . 8
  - 2.2.3 3. VS Code DevContainer . . . 8

3 Project Structure and Conventions 9
- 3.1 The Chapter System . . . 9
  - 3.1.1 Folder Naming . . . 9
  - 3.1.2 File Naming . . . 9
- 3.2 The Meta Directory . . . 9
- 3.3 Assets and Images . . . 10
  - 3.3.1 Remote Assets . . . 10

4 Continuous Integration and Deployment 11
- 4.1 How it Works . . . 11
- 4.2 GitHub Actions Environment . . . 11
- 4.3 Setting it Up . . . 11

Conclusion 12

About the Author 13

Preface

Replace this content with your own preface. A preface usually explains why you wrote the book and what readers can expect to learn.

The `{-}` in the header ensures this section is not numbered in the Table of Contents.

1 Writing Your Book with Markdown

This template uses standard GitHub Flavored Markdown (GFM) with extra support for professional typesetting features.

1.1 Basic Formatting

You can use standard **bold**, *italic*, and `inline code`.

- Itemized lists
- Work just as expected
  - Including nested items

1.1.1 Code Blocks

Code blocks are rendered with the `breezedark` theme:

```
function helloWorld(): string {
  return "Welcome to your new book template!";
}
```

1.1.2 Mathematical Formulas

You can use KaTeX for high-quality math formulas:

$$e^{i\pi} + 1 = 0$$

1.1.3 Diagrams with Mermaid

Render diagrams directly in your book:

1.1.4 Adding Images

Store your images in `assets/images/` and link them using relative paths. This ensures they show up in your Markdown editor preview:

```
![The Author](assets/images/author.png)
```

Output:

Figure 1.1: The Author

2 Automation and the Build Pipeline

One of the core strengths of this template is its automated build pipeline. You focus on the content; the system handles the layout.

2.1 The Build Script

The `build.sh` script is the heart of the system. It performs the following steps: 1. **Gathers Files:** It dynamically finds every markdown file in the `chapters/` directory. 2. **Merges Content:** It creates a temporary `merged.md` file. 3. **Fixes Paths:** It automatically adjusts image paths so they work regardless of where the source file is located. 4. **Runs Pandoc:** It uses Pandoc and the Eisvogel LaTeX template to generate a professional-grade PDF.

2.2 Running the Build

You have three ways to build your book:

2.2.1 1. Local Build

If you have Pandoc and LaTeX installed locally:

```
pnpm run build
```

2.2.2 2. Docker Build

If you don't want to mess with local installations:

```
docker build -t book-builder .
docker run --rm -v $(pwd):/data book-builder
```

2.2.3 3. VS Code DevContainer

The most recommended way! Just open this project in VS Code, and it will offer to reopen it in a container. Everything will be pre-installed and ready to go.

3 Project Structure and Conventions

To keep your book organized and ensure the build script works correctly, it's important to follow the project's folder and file naming conventions.

3.1 The Chapter System

The build script automatically finds all your content by looking inside the `chapters/` folder.

3.1.1 Folder Naming

Each chapter must be in its own folder named `chapter-{n}`: - `chapters/chapter-1/` - `chapters/chapter-2/` - `chapters/chapter-10/`

Note: The script uses "natural sorting," so `chapter-10` correctly comes after `chapter-9`.

3.1.2 File Naming

Inside each folder, you can have multiple Markdown files. The script will concatenate them in alphanumeric order. We recommend using an index: - `1.introduction.md` - `2.deep-dive.md` - `3.summary.md`

3.2 The Meta Directory

The `meta/` directory contains files that aren't part of the main numbered chapters but are essential for a professional book:

1. **`title-copyright.tex`**: A raw LaTeX file for the title and copyright pages. This renders *before* the Table of Contents.
2. **`preface.md`**: The introduction to your book.
3. **`conclusion.md`**: The final chapter.
4. **`about-author.md`**: Your biography and photo.

3.3 Assets and Images

Keep all your media in the `assets/` directory. By convention, images go in `assets/images/`.

When linking images in your chapters, use paths relative to the file so they work in your editor's preview: `![My Image](assets/images/my-image.png)`

3.3.1 Remote Assets

The build pipeline supports remote URLs (e.g. `https://...`) for images anywhere in your book (chapters or metadata). They are automatically downloaded and cached locally during the build process to ensure LaTeX compatibility.

```
![My Image](https://raw.githubusercontent.com/WriteOps/.github/refs/heads/main/logo.png)
```

Result:

Figure 3.1: My Image

4 Continuous Integration and Deployment

The WriteOps Book Template comes pre-configured with GitHub Actions to ensure your book is built and released automatically.

4.1 How it Works

The workflow file is located at `.github/workflows/build-pdf.yml`. It is triggered in two scenarios:

1. **On Push/PR:** Every time you push code to the `main` or `master` branch, the CI pipeline runs a build check to ensure your book still compiles without errors.
2. **On Release:** When you create a new GitHub Release with a tag starting with `v` (e.g., `v1.0.0`), the pipeline builds the final PDF and automatically attaches it to the release as an asset.

4.2 GitHub Actions Environment

The pipeline uses the official Pandoc action and installs a minimal Chromium environment to handle Mermaid diagrams. This ensures that the PDF generated in the cloud is identical to the one you build locally or in Docker.

4.3 Setting it Up

To enable the automated releases: 1. Push your code to GitHub. 2. Go to the **"Actions"** tab to see your first build running. 3. When you're ready to publish, go to **"Releases"** -> **"Create a new release"**. 4. Give it a tag like `v1.0.0` and publish. 5. Wait a few minutes, and your PDF will appear in the release assets!

Conclusion

Summarize your book's main points here and provide some final thoughts for your readers.

About the Author

Share your story here. Let your readers know who you are, your background in the industry, and what drives your passion for writing.

This template allows your text to elegantly wrap around your profile picture, creating a professional look similar to what you'd find in high-end technical books. You can continue writing here and the text will flow naturally around the image box on the left.

Don't forget to include links to your social media, personal website, or other published works!